

Real-Time Two-Way ASL Translator

CP330: EDGE AI 2026

https://github.com/prayanshus/Realtime_ASL_RasPi.git

Team Members:

Liz Maria George Naznin Amirul Haque Ayush Kumar Prayanshu Sharma

M. Tech. (MVL SI), ECE, IISc, Bangalore

Problem Statement

Communication Barrier

- Most hearing people do not understand sign language, creating daily communication barriers.
- Non-signers should not be expected to learn ASL. The solution must bridge the gap without requiring effort from both sides.

Low power and wearable solution

- The solution should involve using a device which is low power, wearable and enables two-way communication between ASL (American Sign Language) users and non signers.

Proposed Solution

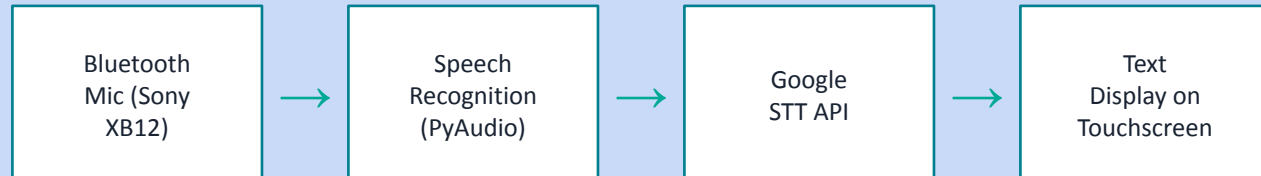
Two-way Communication on Raspberry Pi 5:

- Using **Video-based Word Detection** for converting ASL to speech (to aid speech impairment)
- Using **Audio to Text** for interpreting speech (to aid hearing impairment)

SIGN → SPEECH (Speaker Mode)



SPEECH → TEXT (Listener Mode)



Hardware & Software Setup

Hardware

Component	Specification
Compute	Raspberry Pi 5 (4 GB RAM)
Camera	Pi Camera Module (CSI ribbon)
Display	Official Pi Touchscreen
Audio I/O	Sony SRS-XB12 Bluetooth
OS	Raspberry Pi OS Bookworm 64-bit

Software & Libraries

Library	Purpose
MediaPipe	Hand & pose landmark detection
TensorFlow / TFLite	Model training & on-device inference
Keras Tuner	Neural architecture search
OpenCV	Image processing & capture
Picamera2	Pi camera interface
espeak-ng	Text-to-speech engine
SpeechRecognition	Mic capture + Google STT
Tkinter	GUI framework
NumPy / scikit-learn	Data processing & evaluation

Data Collection & Dataset Preparation

19 Supported Signs

- **Video data collection**

- 180 videos per class (30 fps)
- 3,420 videos for 19 classes in total

- **Collection script features:**

- On-screen sign instructions and countdown timer
- Real-time hand detection warning if hand not visible

- Yes
- No
- Hello
- Please
- Thank you
- Water
- More
- Home
- Good
- Bad
- Come
- Go
- Stop
- Time
- Me
- Food
- Want
- Help

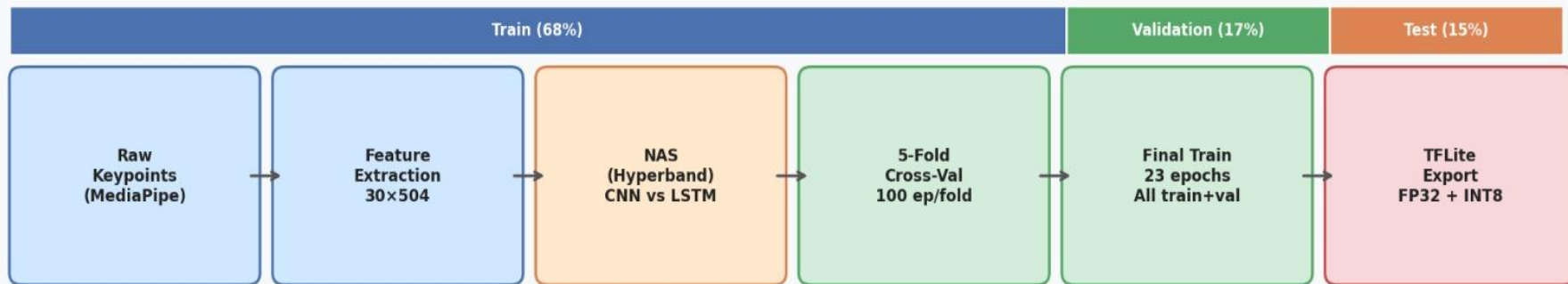
Feature Engineering

- MediaPipe: Open source library for media applications like keypoint extraction
- Used it to extract 33 pose and 21 per-hand landmarks
- Engineered into a 252-dimensional feature vector:
 - Feature groups ensuring invariance to hand size and position
 - Features to capture dynamics in videos

Feature Group	Dims	Description
Pose landmarks	99	33 body keypoints $\times$ 3 (x,y,z) — spatial context
Right hand (norm.)	63	21 keypoints $\times$ 3, wrist-centered & scale-normalized
Left hand (norm.)	63	Same normalization applied to left hand
Fingertip distances (R+L)	20	Pairwise distances between 5 tips, $C(5,2)=10$ per hand
Palm normals (R+L)	6	Cross-product unit vector of palm plane
Inter-hand distance	1	Wrist-to-wrist distance (two-handed signs)
Velocity features	252	Frame-to-frame change (difference) computed for all 252 base features
Total Features	504	Concatenated base and velocity features per frame

Final Input Tensor - (30, 504) - 30 frames $\times$ (252 base + 252 velocity) features

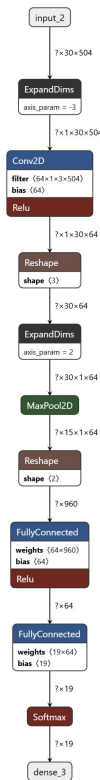
Model Design, Training & Evaluation



Model Design, Training & Evaluation

- **Step 1: Neural Architecture Search (NAS)**
 - **Search Space:**
 - Architecture: CNN (Conv1D) vs LSTM
 - LSTM units / Conv filters: 32–128
 - Dense units: 32 or 64
 - Dropout: 0.2–0.5
 - Learning rate: 1e-3, 1e-4
 - Kernel size (CNN): 3 or 5
- **Step 2: 5-Fold Stratified Cross-Validation**
 - 100 epochs per fold.
 - Measures generalization stability (mean $\pm$ std accuracy).
 - Mean best epoch $\rightarrow$ final training duration.

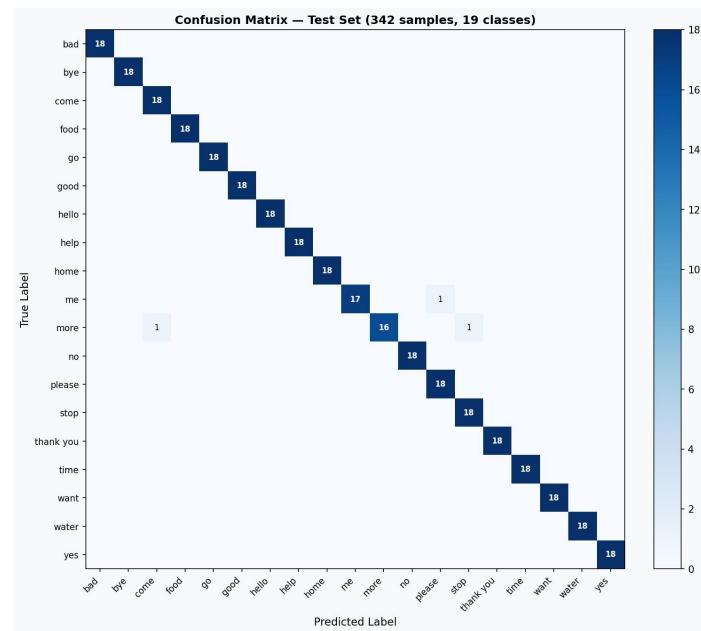
Final Architecture: CNN



Model Compression & Efficiency

Post Training Quantization (PTQ) performed:

Metric	FP32	INT8	Reduction
File size	627 KB	162 KB	3.86× smaller
Total tensor RAM	771.9 KB	252.5 KB	3.06× less
x86 inference latency	0.082 ms	0.023 ms	3.5× faster
Test accuracy	99.12%	99.12%	No degradation



PTQ gave sufficient test accuracy, so did not choose QAT

Model Deployment & On-Device Performance

Deployment Steps

- Train model on PC. Save the following files
 - FP32 + INT8 .tflite files + sign_labels.npy
- Transfer files to Raspberry Pi 5
- Run pi_two_way_translator_V2.py

On-Device Performance

Metric	FP32	INT8
Model Size	628 KB	163 KB
Inference Time	~0.5 ms	~0.5 ms
GUI Frame Rate	20-30 FPS	20-30 FPS
Confidence Threshold	85%	85%

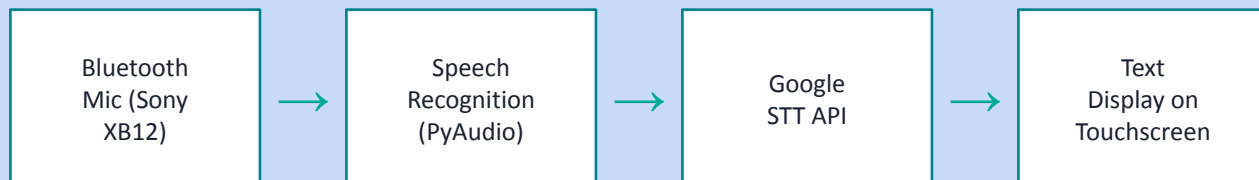
Multi-Threaded Architecture (4 threads):

- **Main Thread:**
 - GUI rendering + camera capture
- **Inference:**
 - TFLite model on dedicated thread
- **Text to speech:**
 - espeak-ng thread
- **Speech to text:**
 - Mic capture + Speech to text conversion

Speech to Text

- **Development challenges:**
 - **VOSK:**
 - Struggled to transcribe full sentences correctly, only worked for words
 - Knowledge Distillation from OpenAI whisper to a small model:
 - Compute limitations. Got high WER (Word Error Rate), so dropped.
 - Will be taken up as future work.
- **Current fallback option:**
 - Google Speech to Text API

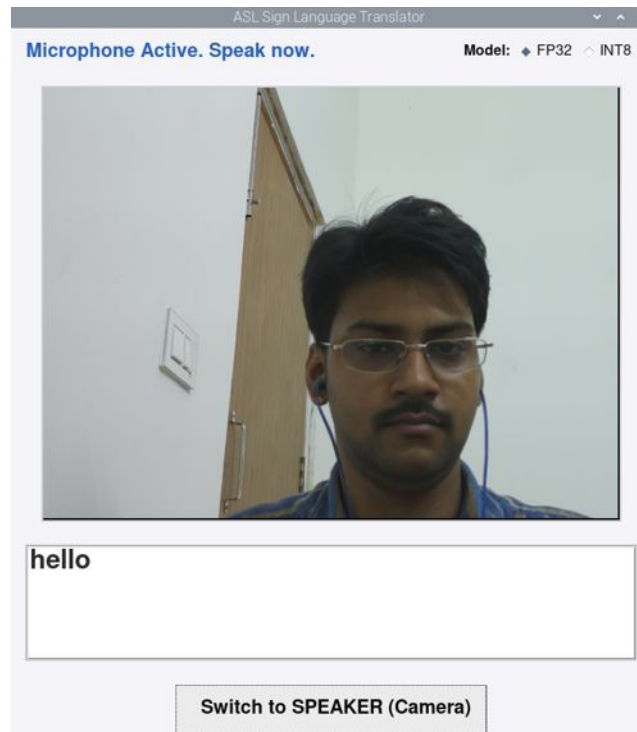
SPEECH → TEXT (Listener Mode)



System Prototype

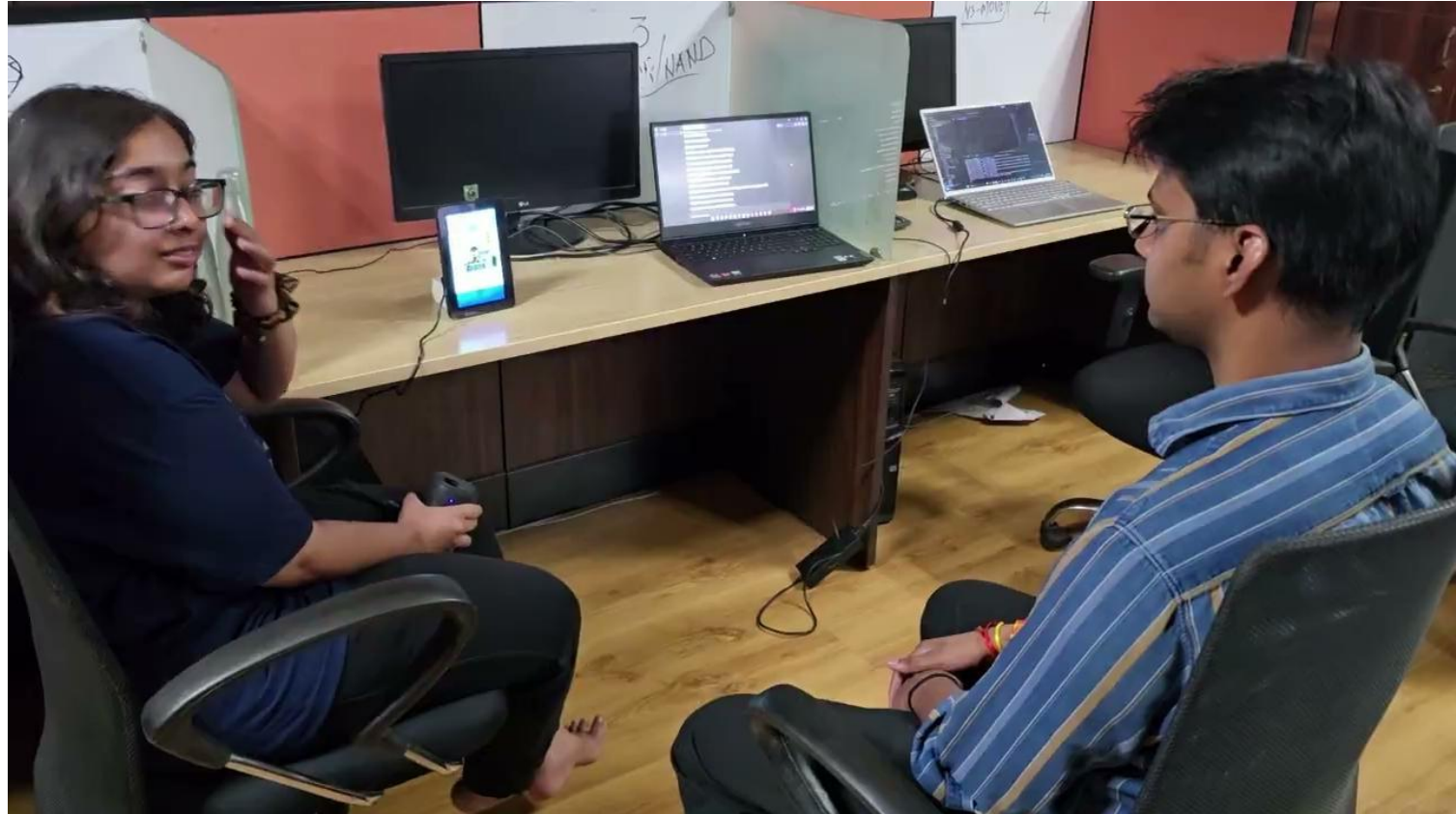


Speaker Mode: Sign → Speech



Listener Mode: Speech → Text

Demonstration



Conclusions & Limitations

Key Outcomes

- Real-time two-way ASL translator running entirely on Raspberry Pi 5 edge hardware
- 19-sign vocabulary with CNN classifier achieving high accuracy (85% confidence threshold)
- Both FP32 and INT8 (PTQ) models deployed with runtime model switching
- Bidirectional: Sign→Speech and Speech→Text in a single application

Limitations

- **Vocabulary:** Can be expanded to cover the entire ASL, ISL
- **Lighting:** Current implementation requires good lighting. Can implement auto-exposure for pi-camera and expand the training dataset with varying lighting
- **Speech-to-Text:** Currently not offline, will be ported to smaller model later.
- **Hardware:** Current setup is for Proof of Concept:
 - Can be easily converted to a wearable hardware

Future Work

Hardware Redesign:

- **Chest-Mounted Box** containing:
 - Raspberry Pi compute module
 - Camera (forward-facing)
 - Microphone + Speaker
 - Battery pack

Software & Model Improvements:

- **Expand Vocabulary**
- **Fully Offline Speech to Text**
 - Replace Google API with on-device distil-whisper
- **Dataset augmentation**
 - Capture training set from signer's POV

- **Wrist-Mounted Display** showing:

- Transcribed speech text
- Mode toggle controls



Visualized by AI

Thank You!

Team Members:

Liz Maria George Naznin Amirul Haque Ayush Kumar Prayanshu Sharma